

Meta-Model Diagnostic: Can a Learned Blend Beat the Crowd?

Date: 2026-07-05 **Author:** Souparneya Chakrabarti **Script:** `ml/meta_model.py` **Output:** `ml/meta_model_results.json` **Status:** Diagnostic only. No pricing code or live estimates were touched. Follows the same precedent as `ml/platt_diagnostic.py` — fit, validate out-of-sample, and only recommend deployment if the improvement survives.

1. Motivation

The project currently blends `our_estimate` and `crowd_estimate` using hand-tuned, situational rules (RULE1’s 50/50 average, RULE8’s 65/35 shrink toward 0.5, RULE12’s pull-to-crowd trigger, etc.). Each rule was derived by tracing a specific win or loss back to its cause — useful, but ad hoc, and each one only covers the narrow situation that produced it.

The question this experiment asks: **can a single learned model, given the structural features already sitting in the master dataset (Elo gap, rest days, squad value gap, draw probability, which rules fired), produce a better-calibrated final probability than the existing hand-tuned rules — without needing any new data acquisition?** This is “Problem A” from the ML discussion: a meta-model trained entirely on data the project already has, as distinct from “Problem B” (acquiring external club-level data to replace the domain models themselves).

2. Data Used

Source: `datasets/master_question_dataset.csv` — the project’s authoritative flat file, one row per (match, question) pair, built by `datasets/build_master_dataset.py` from raw per-match JSON files. Full column dictionary: `datasets/MASTER_DATASET_DICTIONARY.md`.

2.1 Filtering to a usable sample

The raw file has 580 rows and 105 columns, but not every row is usable for this experiment. A row was kept only if all three of these were present:

```
usable = df[
    (df["actually_submitted"] == True)
    & df["our_estimate"].notna()
    & df["crowd_estimate"].notna()
    & df["outcome"].notna()
]
```

This yields **404 usable rows across 42 unique matches**, spanning 2026-06-11 to 2026-06-30. Note this is *group-stage-only* — the master dataset has not yet had the knockout-stage matches (stored separately under `matches/{Team1}_vs_{Team2}/`) merged in, so this experiment does not see AUS-EGY, ARG-CPV, COL-GHA, CAN-MOR, etc. That gap is a known, free source of additional rows for a re-run (see §7).

2.2 What the data looks like

First 6 rows of the usable sample, selected columns:

match	date	q#	question	our_est	crowd_est	outcome	bp	elo_diff	rest_days	fifa_diff	fifa_rank	draw_prob
KOR-CZE	2026-06-11	Q1	Both teams score AND 3+ total goals?	0.40	0.38	1	+6.44	73.6	1	25	43	0.303
KOR-CZE	2026-06-11	Q2	Penalty kick awarded?	0.42	0.33	0	−3.20	73.6	1	25	43	0.303
KOR-CZE	2026-06-11	Q3	S. Korea more 2H SOT than Czechia?	0.60	0.55	1	+7.21	73.6	1	25	43	0.303
KOR-CZE	2026-06-11	Q4	Will South Korea win?	0.46	0.51	1	−0.08	73.6	1	25	43	0.303
KOR-CZE	2026-06-11	Q5	Son Heung-min scores?	0.38	0.40	0	+6.52	73.6	1	25	43	0.303
KOR-CZE	2026-06-11	Q6	Schick 1+ SOT?	0.65	0.67	0	+5.91	73.6	1	25	43	0.303

Note that every question within a match shares the same match-level context columns (`elo_diff`, `rest_days_diff`, `fifa_ranking`, `draw_probability`) — this is the source of the clustering that shapes the validation design in §4.2.

2.3 Missingness (within the 404-row usable sample)

Column	Missing	Notes
<code>elo_diff</code>	15 / 404	Teams without a resolvable Elo pairing

Column	Missing	Notes
squad_value_a_eur / _b_eur	59 / 404	5 Transfermarkt-missing nations (CIV, CUR, CDR, HAI, CPV)
fifa_ranking_2025_a / _b	29 / 59	Same source gap
draw_probability	15 / 404	Tied to the Elo gap
question_category	327 / 404 (81%)	Too sparse to use as a feature — dropped
edge_rank, direction	385–387 / 404	Too sparse — dropped
rest_days_diff, rule_fired_count	0 / 404	Fully populated

question_category, edge_rank, direction, and is_player_prop (all-False in this sample) were dropped from the feature set entirely rather than imputed — 80%+ missingness on a categorical column isn’t something imputation fixes.

Fifteen of the eighteen named rules (rule2–rule4, rule6, rule9, rule11, plus low-firing ones) either never fire or fire fewer than 10 times in this sample; only 9 rule flags cleared a 10-firing threshold and were kept as binary features (see §3).

3. Feature Engineering

Built in `engineer_features()` in `ml/meta_model.py`:

Feature	Definition
our_estimate, crowd_estimate	Raw values
gap	<code>our_estimate - crowd_estimate</code>
abs_gap	<code>\ gap\ </code>
elo_diff	As stored
rest_days_diff	As stored
squad_value_diff_m	<code>(squad_value_a_eur - squad_value_b_eur) / 1e6</code>
fifa_ranking_diff	<code>fifa_ranking_2025_b - fifa_ranking_2025_a</code> (positive = team A ranked better)
draw_probability	As stored
rule_fired_count	As stored
rule{1,5,7,8,10,12,13,14,15}_fired	Binary, kept only if fired 10 times in-sample

Missing values for the continuous features were median-imputed with a missing-indicator column added (`SimpleImputer(add_indicator=True)`), so “this value was missing” is itself available as a signal rather than silently disappearing. **19 features total on 404 rows** — already a thin ratio, which matters for interpreting the results in §5.

4. Methodology

4.1 Models

Two candidates, both deliberately regularized given the sample size:

- **Logistic regression** — `StandardScaler + LogisticRegression(C=0.3)` (strong L2 penalty), inside an `sklearn Pipeline` with the imputer.
- **Histogram gradient boosting** — `sklearn's HistGradientBoostingClassifier(max_depth=2, max_iter=60, learning_rate=0.05, l2_regularization=1.0)`. Shallow trees, few boosting rounds, explicit L2 term — handles missing values natively.

4.2 Baselines

Three zero-parameter references, computed on the exact same held-out rows as each model for a fair comparison:

1. `crowd_alone` — Brier score of `crowd_estimate` against outcome
2. `our_estimate_alone` — Brier score of `our_estimate` against outcome
3. `naive_50_50_avg` — Brier score of `0.5·our_estimate + 0.5·crowd_estimate`

These have no fitted parameters and therefore cannot overfit — any candidate model has to beat them from a standing start.

4.3 Validation — two schemes, deliberately redundant

Following the project's standing rule against random k-fold on temporally-ordered data:

(a) **Time-ordered walk-forward.** Matches sorted chronologically; first 21 matches (~half) used as a burn-in training set; then, expanding the training window one match at a time, the model predicts the next match's questions out-of-sample. This exactly mirrors `model/backtest_harness.py`'s philosophy — no lookahead.

(b) **Grouped 6-fold cross-validation, grouped by match.** A robustness check: since only 21 matches ever get to be “test” data under scheme (a), this scheme lets every match serve as held-out data at some point, while `GroupKFold` still guarantees no single match's questions are split across train and test (which would leak match-level context).

A model is only considered promising if it beats the best baseline under **both** schemes by a non-trivial margin.

4.4 Converting Brier improvement into RBP terms

Since `rbp = (crowd_brier_term - our_brier_term)`, the proportionality constant **S** was fit directly from the project's own recorded data (386 rows with a known `rbp` value), regressed through the origin:

```
crowd_term = (crowd_estimate - outcome) ** 2
our_term    = (our_estimate - outcome) ** 2
S = sum(x * y) / sum(x * x)    # x = crowd_term - our_term, y = rbp
```

This gave **S = 101.3 RBP per unit of Brier improvement** — an empirical scale factor specific to this project's JTC platform, letting any Brier improvement be read off directly as an expected RBP swing.

5. Results

5.1 Baselines (full sample, in-sample by construction — no fitting involved)

Baseline	Brier score
<code>crowd_alone</code>	0.2156
<code>our_estimate_alone</code>	0.2179
<code>naive_50_50_avg</code>	0.2139

(Consistent with the RMD’s earlier finding that raw crowd Brier already beats raw project Brier — the crowd’s unweighted number is a strong reference, not a weak one.)

5.2 Walk-forward out-of-sample (`n_test` = 208, matches 22–42)

Model	Brier	Best baseline (same rows)	Improvement	Est. RBP equivalent
Logistic regression	0.2545	0.2157	−0.0388	−817.6
HGB	0.2227	0.2157	−0.0070	−147.9

5.3 Grouped k-fold out-of-sample (`n_test` = 404, all matches)

Model	Brier	Best baseline	Improvement	Est. RBP equivalent
Logistic regression	0.2569	0.2139	−0.0430	−1760.3
HGB	0.2315	0.2139	−0.0176	−721.1

Both models, under both validation schemes, are worse than simply trusting the crowd. The logistic regression is worse than HGB in both schemes, suggesting the linear model is more sensitive to the 19-feature/~200-400-row ratio than the shallow tree ensemble is, but neither clears the bar.

5.4 Feature signal (fit on full data, for interpretability only — not the validated models above)

Standardized logistic regression coefficients, ranked by magnitude:

Feature	Std. coefficient
<code>crowd_estimate</code>	+0.590
<code>fifa_ranking_diff</code>	+0.450
<code>our_estimate</code>	+0.350
<code>rule1_fired</code>	−0.307
<code>squad_value_diff_m</code>	−0.209
<code>elo_diff</code>	−0.198

Feature	Std. coefficient
rule15_fired	+0.171
gap	−0.147
draw_probability	−0.106

HGB permutation importances (top 6, all others 0):

Feature	Importance (mean Brier degradation when shuffled)
crowd_estimate	0.0594
fifa_ranking_diff	0.0174
our_estimate	0.0069
squad_value_diff_m	0.0054
draw_probability	0.0035
elo_diff	0.0029

Every named rule flag (rule1–rule15) contributes essentially nothing beyond the continuous features in the tree model — a genuinely useful negative finding independent of the overall verdict: the rules may be encoding less unique information than the raw match-context numbers already do.

6. Interpretation

`crowd_alone` is a zero-parameter reference — it cannot overfit because nothing is fitted. Beating it requires a model to extract real signal from 19 features using 208–404 rows clustered into 21–42 matches (the true unit of independent variation, since every question in a match shares the same context columns). That is not enough data for either candidate to reliably separate signal from noise, even under heavy regularization. This is the same failure mode the project already documented for Platt scaling at $n=246$ (“applying the correction degrades out-of-sample Brier... decision: do not apply yet”) — this experiment reaches the same kind of honest negative result, on a different model class.

This is not a bug in the script or a wrong modeling choice; it is what “not enough data yet” looks like when measured properly, instead of being asserted.

7. Decision and Next Steps

Decision: do not deploy either model. Per the script’s own output:

Improvement is small, inconsistent between walk-forward and grouped k-fold, or negative for at least one model/scheme. Do NOT deploy. Matches the Platt-diagnostic precedent: revisit once the master dataset includes the knockout-stage matches (more rows, more matches) and re-run this script unchanged.

Concrete next steps, in order of cost:

1. **Free data, no acquisition needed:** fold the knockout-stage matches already sitting in `matches/{Team1}_vs_{Team2}/` (AUS-EGY, ARG-CPV, COL-GHA, CAN-MOR, MEX-ECU, and others) into `master_question_dataset.csv`, then re-run `ml/meta_model.py` unchanged. This alone should take the sample from 42 to 50+ matches.
 2. **Cheap experiment, no new data:** re-run with a much smaller, hand-picked feature set (e.g., just `crowd_estimate`, `elo_diff`, `fifa_ranking_diff`) to test whether the failure is “too many features for the data” rather than “no learnable signal exists” — 19 features on ~400 rows may simply be over-specified regardless of regularization strength.
 3. **Longer term:** this result is itself an argument for prioritizing “Problem B” (external club-level data acquisition) — the meta-model approach is sound in principle, but this dataset’s size is the binding constraint, and only new data (not better modeling) resolves that.
-

Appendix: Reproducing This

```
python3 ml/meta_model.py
```

Reads `datasets/master_question_dataset.csv`, writes `ml/meta_model_results.json`. No arguments, no side effects on any other file.